

Islamic University of Technology

Lab 5: Dynamic Programming Engineering & Live Profiling

CSE 4810: Algorithm Engineering
OBST

ASH, ABT(PT)
July 16, 2026

General Instruction

This lab consists of **four independent tasks**. Unlike previous labs, written derivations are replaced by **live profiling, execution, and verbal defense**. For every task you must:

- Step 1.** Complete the `// TODO` sections in the provided C++ source file.
- Step 2.** Compile and run the program, feeding it the exact stress-test inputs specified.
- Step 3.** Produce the requested live artifact — a runtime plot, a memory log, or a comparison count — and keep your terminal/IDE open.
- Step 4.** Be ready to defend your engineering choices verbally using the artifact on screen; no written report is collected.

Files Provided

`OBST.cpp` Standard OBST skeleton
`plotter.py` Plots runtime/memory data

Task: Structural Engineering of an Optimal BST (OBST)

Objective: Mitigate search latencies for non-uniform data distributions by engineering away a hidden redundant-computation bottleneck.

Variant	Strategy	Why It Matters
obst_naive	Recomputes the frequency-range sum <code>freq_sum(i, j)</code> from scratch with a loop for <i>every</i> root candidate r , even though it never changes with r	Correct, but wasteful. That redundant $O(n)$ work inside the root loop inflates the true recurrence from $O(n^3)$ to $O(n^4)$.
obst_optimized	Precomputes a prefix-sum array once, so <code>freq_sum(i, j)</code> is an $O(1)$ lookup instead of a recomputed loop	Removes the hidden redundant factor , restoring the textbook $O(n^3)$ bound — no monotonicity argument required.

0.1 TODO 1: Implement `obst_naive(keys, probabilities)`

0.2 TODO 2: Implement `obst_optimized(keys, probabilities)`

0.3 TODO 3: Implement `simulate_obst_lookups(n, const, probabilities, root, trials)`

Sanity Check — Expected Output

```
1 --- OBST SANITY CHECK (n=4) ---
2 Naive      : 1.7 (Expected 1.7)
3 Optimized  : 1.7 (Expected 1.7, should match Naive)
4 -----
```

Live Experimentation Setup

Runs to Perform

- **The Execution Run:** Run both variants for a heavily skewed search distribution (3 "hot" keys holding 90% of the search probability) at $n = 50, 100, 150$.
- **The Stress Test:** At $n = 400$, run both variants again. Naive should take on the order of seconds; optimized should still be near-instant. Kill the naive run if it exceeds 30 seconds on your machine.
- **The Benchmark:** Build a balanced tree (e.g., `std::set/std::map`, which is red-black-tree backed) alongside the OBST. Simulate 100,000 random lookups weighted by the specified probabilities and track the total number of comparisons made by each structure.

Compile & Run

```
1 g++ -O2 -o obst OBST.cpp
2 ./obst
3 python3 plotter.py data_obst.txt
```

Core Engineering Defense (Verbal Check)

- **The Curve Break:** Point to your naive-vs-optimized timing plot and explain why the *ratio* between the two curves keeps growing as n increases, rather than staying at a fixed multiple — that widening ratio, not just "optimized is faster," is the actual signature of an $O(n^4)$ vs. $O(n^3)$ difference.
- **Find the Redundancy:** Walk through `obst_naive`'s root loop line by line and point to the exact statement that gets re-executed unnecessarily on every iteration of `r`. Explain why that single design choice, not the algorithm's structure, is responsible for the extra factor of n .
- **The Cost Trade-off:** Show the exact reduction in lookup comparisons from the benchmark. Explain why a static, skewed OBST is structurally superior to a standard balanced tree when access frequencies are highly unequal.

The instructor may ask you to move the `freq_sum` computation from `obst_optimized` back inside the `r` loop (i.e., turn it back into the naive version) and re-run the $n = 150$ timing. You must observe the timing regression, then explain — without invoking Knuth's theorem or any root-monotonicity argument — purely in terms of "how many times is this line executed," why moving one computation outside a loop changed the asymptotic complexity class.